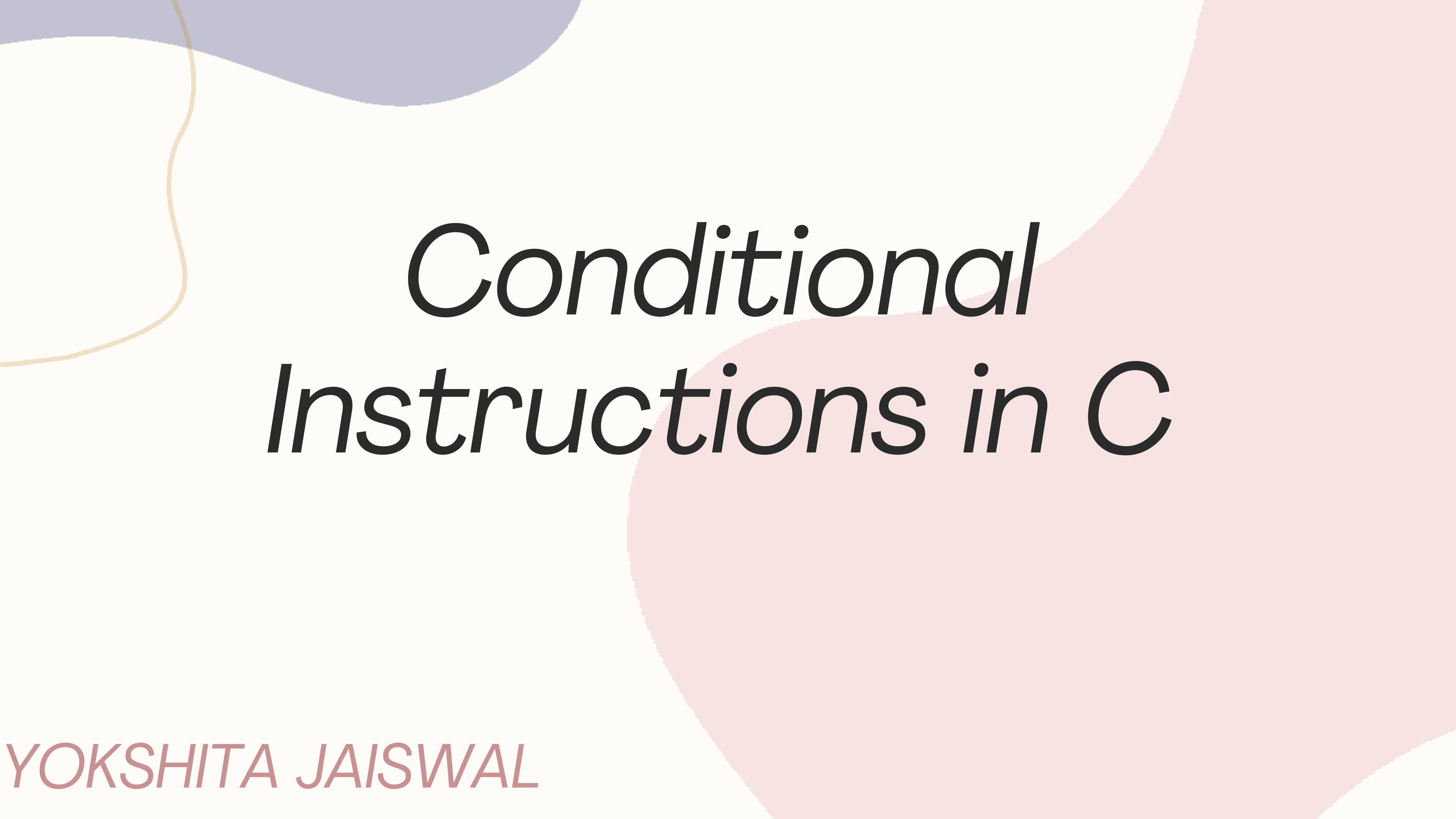




Conditional Instructions in C

YOKSHITA JAISWAL

Introduction to Conditional Instructions

Conditional instructions in C allow a program to make decisions and execute specific blocks of code based on conditions. These instructions enable a program to alter its flow rather than executing all statements sequentially.

Conditional instructions allow decision-making in programs based on conditions. They execute specific code blocks when a condition evaluates to true.

Why Use Conditional Statements?



They help in making decisions within a program.

Allow different execution paths based on conditions.

Improve program efficiency by controlling flow dynamically.

Conditional statements enable dynamic user interaction by responding to user inputs.

TYPES OF CONDITIONAL STATEMENT

 Topics Covered

1

if Statement

2

if-else Statement

3

Nested if

4

if-else-if Statement

5

Switch Case

6

Ternary Operator (?:)

IF Statement in C

The if statement is the simplest conditional instruction in C. It is used to execute a block of code only if a given condition is true. If the condition is false, the program skips that block and continues execution.

CODE:

```
#include <stdio.h>
int main() {
    int number = 10;
    if (number > 5) {
        printf("The number is greater than 5.");
    }
    return 0;
}
```


if-else Statement

The if-else statement is an extension of the if statement. It allows the program to execute one block of code if a condition is true and a different block if the condition is false.

CODE:

```
#include <stdio.h>
int main() {
    int number = 10;

    if (number > 5) {
        printf("The number is greater than
5.");
    } else {
        printf("The number is 5 or less.");
    }

    return 0;
}
```

Nested if Statement

A nested if statement is an if statement inside another if or if-else statement. It is used when we need to check multiple conditions in a hierarchical manner.

CODE:

```
#include <stdio.h>
int main() {
    int age = 20;
    char nationality = 'I'; // I for Indian

    if (age >= 18) {
        if (nationality == 'I') {
            printf("You are eligible to vote in India.");
        }
    }

    return 0;
}
```

if-else-if Ladder

The if-else if ladder is used when we need to check multiple conditions sequentially. It allows a program to test multiple conditions one by one until one condition is true. If none of the conditions are true, the else block executes.

CODE:

```
#include <stdio.h>
int main() {
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);

    if (marks >= 90) {
        printf("Grade: A");
    } else if (marks >= 75) {
        printf("Grade: B");
    } else if (marks >= 50) {
        printf("Grade: C");
    } else {
        printf("Failed");
    }

    return 0;
}
```


Switch Case Statement

The if statement is the simplest conditional instruction in C. It is used to execute a block of code only if a given condition is true. If the condition is false, the program skips that block and continues execution.

SYNTAX:

```
switch(expression) {  
    case value1:  
        // Code block  
        break;  
    case value2:  
        // Code block  
        break;  
    default:  
        // Default block if no case matches  
}
```

Ternary Operator

(?:)

The ternary operator (?:) is a short-hand for if-else statements in C. It provides a concise way to evaluate conditions and return values in a single line of code.

SYNTAX:

condition ? expression1 : expression2;

EXAMPLE:

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    (num % 2 == 0) ? printf("Even") : printf("Odd");

    return 0;
}
```

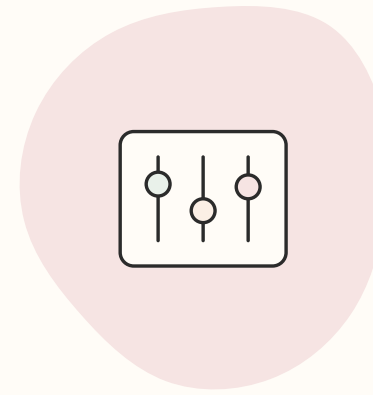
Difference Between if-else and switch-case in C

if-else
Statement

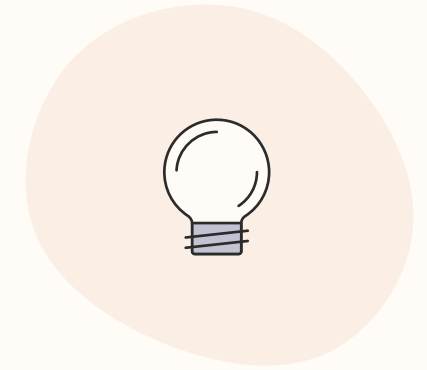
SWITCH-CASE

Execution	Data Types	Use of Default	Efficiency	Code Readability
Checks all conditions sequentially	Supports all data types (int, float, char, string, etc.)	Requires else for default execution	Slower for multiple conditions	Less readable if too many conditions
Directly jumps to the matching case	Works only with int and char (not float or string)	Uses default for unmatched cases	Faster for multiple fixed values	More structured & readable for multiple cases

Common Mistakes



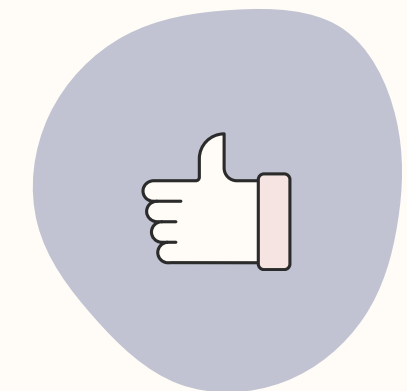
Missing Braces in
if-else Statements



Using Assignment
(=) Instead of
Comparison (==)

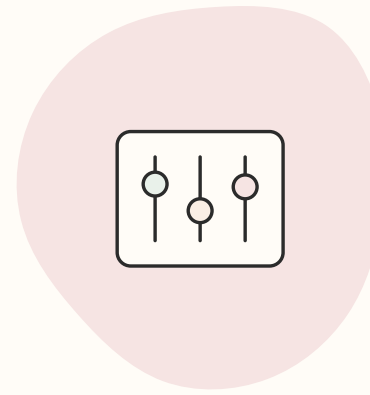


Forgetting break in
switch-case

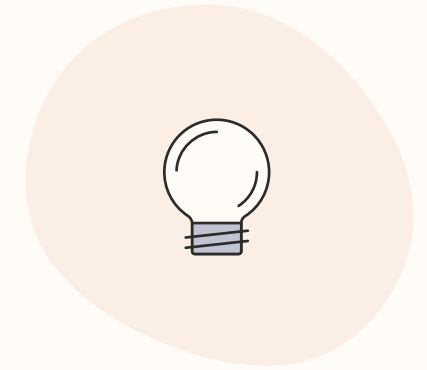


Nested if without
Proper Indentation

BEST PRACTICES



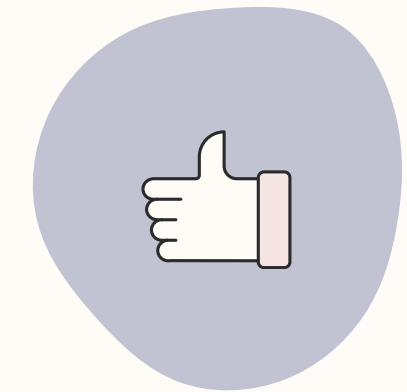
Always Use Braces
{ Even for Single
Statements



Use Ternary
Operator for
Simple Conditions



Always Use Default
in switch-case



Optimize Multiple
Conditions Using
Logical Operators

Real-Life Applications in C

B

Banking System
(ATM
Transactions)

T

Traffic Light
System

G

Grading System
(School/College
Results)

O

Online Shopping
(Discount System)

L

Login
Authentication
System

R

Railway Ticket
Booking System